

UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

José Alberto Manzanilla Alfaro

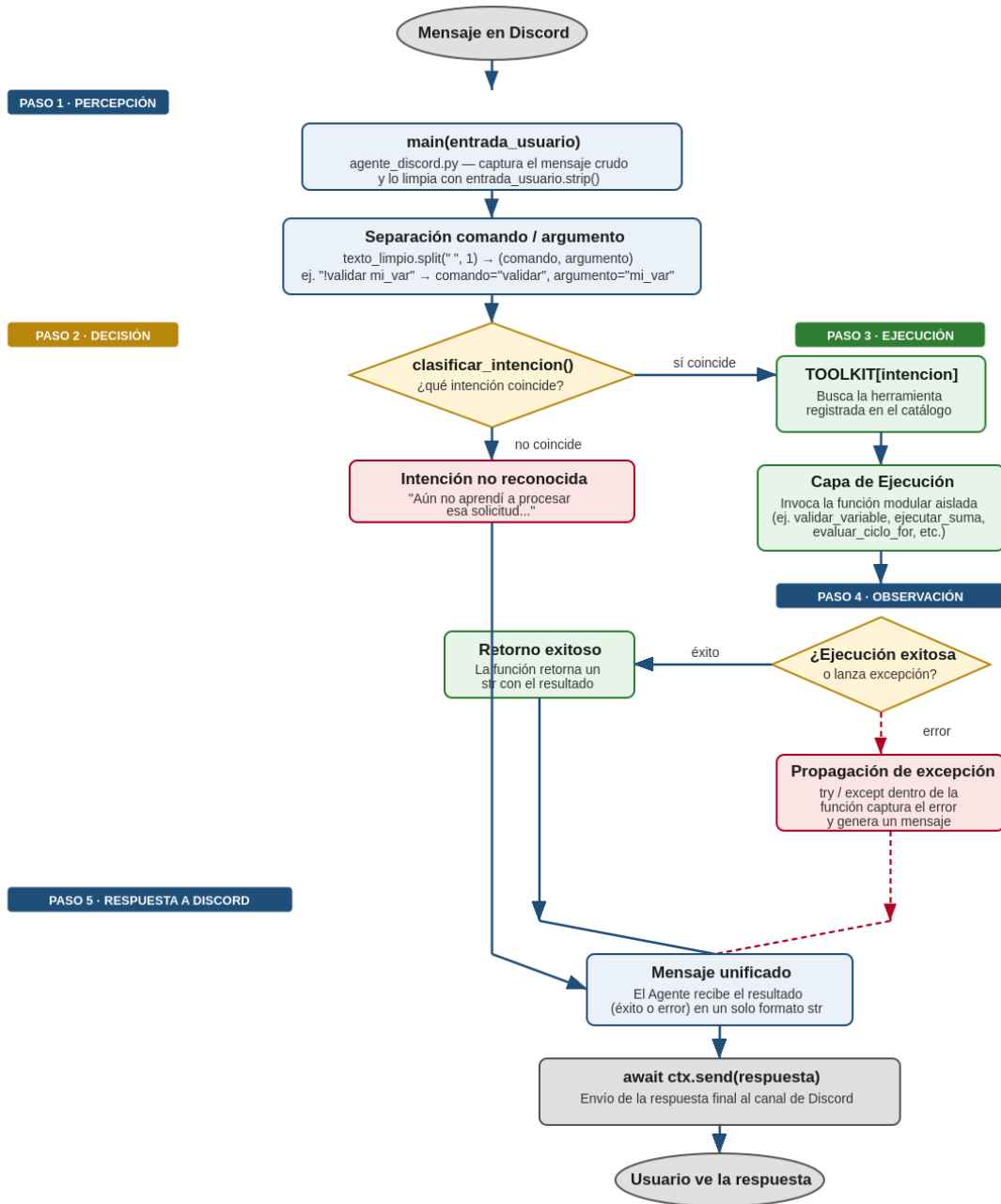
Ingeniería en IA

Programación Estructurada

Diagrama de Flujo del Agente (Arquitectura Completa)

En este diagrama muestro el recorrido completo que hace mi Agente de Discord, desde que un usuario escribe un mensaje en el servidor hasta que el bot le contesta de vuelta en el canal. Lo armé en cinco etapas porque así me resultaba más fácil seguir la lógica, y de paso cada etapa cae justo en uno de los cinco puntos que pide este entregable. La idea principal es que se note que el bot no resuelve todo en un solo bloque de código: primero percibe el mensaje, luego decide qué hacer, después ejecuta la herramienta que toca y al final responde. Ese desacople entre percepción, decisión y ejecución es justo lo que hace que se comporte como un agente y no como un chatbot lineal.

- **Paso 1 (Percepción):** Aquí entra el mensaje tal cual lo escribió el usuario. La función `main(entrada_usuario)` lo recibe, lo limpia con `strip()` y luego separa el comando del argumento usando `split(" ", 1)`. Por ejemplo, `"!validar mi_var"` me queda partido en el comando `"validar"` y el argumento `"mi_var"`, que es justo lo que necesito para saber qué me están pidiendo.
- **Paso 2 (Decisión):** En vez de armar una cadena enorme de `if-elif-else`, dejo que `clasificar_intencion()` revise el texto y me diga qué intención coincide. Esa es la pieza que me permite tratar al bot como un agente y no como un chatbot rígido. Si ninguna intención encaja, el bot no se cae: responde de buena manera que todavía no aprendió a procesar esa solicitud.
- **Paso 3 (Ejecución):** Una vez que ya sé la intención, el agente la busca en el catálogo `TOOLKIT[intencion]` y desde ahí desvía el flujo hacia la función modular que toca (por ejemplo `validar_variable` o `ejecutar_suma`). Cada función trabaja por su cuenta, aislada del resto, así que puedo agregar o quitar herramientas del catálogo sin tener que tocar la lógica central del bot.
- **Paso 4 (Observación):** Aquí la función hace su trabajo por dentro. Si todo sale bien, regresa un `str` con el resultado; si algo falla, su propio bloque `try-except` atrapa el error y lo convierte en un mensaje claro, en lugar de dejar que la excepción se propague y tumbé al bot. Este es el punto donde aplico el manejo defensivo de errores del que hablé en los entregables anteriores.
- **Paso 5 (Respuesta):** No importa si el camino fue de éxito o de error: los dos terminan en un mismo formato de mensaje. Ese mensaje final es el que el bot manda de regreso al canal de Discord con `await ctx.send(respuesta)`, y es lo que el usuario termina viendo en su pantalla. Con esto se cierra el ciclo completo del agente.



Leyenda

- Capa de Percepción / E-S
- Capa de Decisión (condicional / clasificación)
- Capa de Ejecución (función modular del Toolkit)
- Manejo de error / excepción
- Flujo normal
- Flujo de error / excepción

Correspondencia con los puntos solicitados:

1. Captura y separación → Paso 1 (Percepción)
2. Estructura condicional / mapeo de comando → Paso 2 (clasificar_intencion + TOOLKIT)
3. Desvío a función modular → Paso 3 (Capa de Ejecución)
4. Procesamiento interno, éxito o excepción → Paso 4 (Observación)
5. Envío de respuesta a Discord → Paso 5